

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Damián Kružliak

Genetic algorithms for evolving chess evaluation functions

Department of Theoretical Computer Science and Mathematical Logic

Supervisor of the bachelor thesis: RNDr. Jakub Bulín, Ph.D.
Study programme: Computer Science
Study branch: Artificial Intelligence

Prague 2026

I declare that the following AI tools have been used in preparation of this thesis in the ways indicated:

- ChatGPT in the period 1/6/2026 – 16/7/2026, usage description: used mainly as a linguistic, stylistic, and consultation tool. It helped with formulating selected passages, improving English wording, checking consistency between chapters, and suggesting possible edits to the structure.

Having used the above AI tools, I have thoroughly revised and edited the content as necessary and take full responsibility for the final form of the thesis.

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

I would like to thank my supervisor, RNDr. Jakub Bulín, Ph.D., for his guidance and valuable advice throughout the completion of this thesis.

I would also like to thank my girlfriend and my family for their constant support, encouragement, and belief in me.

Title: Genetic algorithms for evolving chess evaluation functions

Author: Damián Kružliak

Department: Department of Theoretical Computer Science and Mathematical Logic

Supervisor: RNDr. Jakub Bulín, Ph.D., Department of Theoretical Computer Science and Mathematical Logic

Abstract: This thesis presents an evolutionary approach to tuning the evaluation function of a chess engine, based on the method proposed by David et al. [1]. The engine uses a fixed set of evaluation features, while their numerical weights are optimized by a genetic algorithm. In the first stage, the population is trained on positions extracted from recorded games. In the second stage, selected individuals continue evolving through self-play. The implementation also provides a playable chess engine with support for the Universal Chess Interface. The experiments show that the system can produce a working set of automatically tuned evaluation parameters, but also point to limitations of exact move-prediction fitness, self-play evaluation, and the fixed search procedure.

Keywords: computer chess, evolutionary algorithms, genetic programming

Název práce: Genetické algoritmy pro evoluci šachových hodnoticích funkcí

Autor: Damián Kružliak

Katedra: Katedra teoretické informatiky a matematické logiky

Vedoucí bakalářské práce: RNDr. Jakub Bulín, Ph.D., Katedra teoretické informatiky a matematické logiky

Abstrakt: Tato práce představuje evoluční přístup k ladění evaluační funkce šachového enginu, založený na metodě popsané v [1]. Engine používá pevně danou sadu evaluačních příznaků, zatímco jejich číselné váhy jsou optimalizovány genetickým algoritmem. V první fázi je populace trénována na pozicích získaných ze zaznamenaných partií. Ve druhé fázi vybraní jedinci pokračují v evoluci pomocí self-play. Implementace zároveň poskytuje hratelný šachový engine s podporou rozhraní Universal Chess Interface. Experimenty ukazují, že systém dokáže vytvořit funkční sadu automaticky naladěných parametrů evaluační funkce, ale zároveň poukazují na omezení přesné predikce tahů, self-play evaluace a pevně daného prohledávání.

Klíčová slova: počítačový šach, evoluční algoritmy, genetické programování

Contents

Introduction **7**

1 Background and Related Work **9**
1.1 Evolutionary Algorithms . . . 9
1.1.1 Individuals, Genomes, and Populations . . . 9
1.1.2 Fitness Function . . . 9
1.1.3 Selection . . . 10
1.1.4 Genetic Operators . . . 10
1.1.5 Genetic Algorithm . . . 10
1.1.6 Genetic Programming . . . 11
1.2 Game Playing and Chess Engines . . . 11
1.2.1 Games . . . 11
1.2.2 Chess . . . 12
1.2.3 Chess Data Formats and Interfaces . . . 13
1.2.4 Evaluation Functions . . . 13
1.3 Chess Engine Optimization . . . 14
1.3.1 Manual and Parameter-Based Tuning . . . 14
1.3.2 Temporal-Difference Learning . . . 15
1.3.3 Self-Learning Evolutionary Players . . . 15
1.3.4 Genetic Programming for Chess . . . 15
1.3.5 Expert-Driven Learning . . . 16

2 Evolutionary Design **17**
2.1 Evaluation Function Design . . . 17
2.1.1 Score Convention . . . 17
2.1.2 Hand-Engineered Features . . . 17
2.2 Genome Representation . . . 18
2.2.1 Schema-Based Bit Encoding . . . 18
2.2.2 Decoding Genomes into Parameters . . . 19
2.2.3 Genetic Operators . . . 19
2.3 Stage 1: Evolution from Recorded Games . . . 20
2.3.1 Training Positions . . . 20
2.3.2 Fitness Computation . . . 20
2.3.3 Train-Test Split . . . 21
2.4 Stage 2: Self-Play Evolution . . . 21
2.4.1 Initial Population . . . 21
2.4.2 Self-Play Fitness . . . 21
2.4.3 Search Budget . . . 22
2.4.4 Hall of Fame . . . 22
2.4.5 Opening Positions . . . 23

3 Implementation **24**
3.1 Project Structure . . . 24
3.2 Configuration and Application Entry Points . . . 25
3.3 Chess Engine Core . . . 25

3.3.1 Board Adapter . . . 25
3.3.2 Search . . . 26
3.4 Evaluation and Genome Decoding . . . 27
3.4.1 Evaluation Parameters . . . 27
3.4.2 Parametric Evaluator . . . 27
3.4.3 Genome Decoding . . . 27
3.5 Data Preparation . . . 28
3.6 Training Implementation . . . 28
3.6.1 Stage 1 Driver . . . 28
3.6.2 Stage 2 Driver . . . 29
3.6.3 Game and Match Execution . . . 29
3.6.4 Opening Positions . . . 30
3.7 Checkpointing and Metrics . . . 30
3.8 Testing and Validation . . . 30
3.9 Interactive and UCI Testing . . . 31

4 Experiments and Results **32**
4.1 Experimental Setup . . . 32
4.1.1 Stage 1 Setup . . . 32
4.1.2 Stage 2 Setup . . . 33
4.2 Stage 1 Results . . . 33
4.2.1 Training Behaviour . . . 33
4.2.2 Test Accuracy . . . 34
4.3 Stage 2 Results . . . 35
4.3.1 Self-Play Fitness . . . 35
4.4 Final Organism Selection . . . 36
4.5 Final Organism . . . 37
4.5.1 Selected Organism . . . 37
4.5.2 Move-Prediction Accuracy . . . 38
4.6 Discussion and Limitations . . . 39

Conclusion **41**

Bibliography **43**

List of Figures **45**

List of Tables **46**

List of Abbreviations **47**

A Attachments **48**
A.1 Electronic Attachments . . . 48

Introduction

Chess has been an important domain in artificial intelligence since the early development of computer science. It is a deterministic two-player game with perfect information, clearly defined rules, and a large search space. These properties make it suitable for studying search, evaluation, learning, and optimization. Early work by Shannon already described the basic idea of a chess-playing program based on game-tree search and heuristic evaluation [2]. Later systems, such as Deep Blue, showed that a combination of efficient search, specialized engineering, and a strong evaluation function could reach world-champion level play [3].

A classical chess engine does not normally search the complete game tree. Instead, it searches a limited number of moves ahead and then estimates the value of the resulting positions. This estimate is produced by an evaluation function. The value should reflect how favorable the position is for White or Black. A positive score usually means an advantage for White, a negative score means an advantage for Black, and a score close to zero means an approximately equal position.

Designing a good evaluation function is difficult. It is not enough to count material only, because many positions with equal material can differ greatly in true evaluation. Figure 1 shows a simple example: both sides have a rook and one pawn, but White's advanced passed pawn and active rook make the position more favorable for White. A useful evaluation function should therefore consider not only material, but also features such as king safety, pawn structure, mobility, passed pawns, open files, or piece placement. These features must then be assigned numerical weights. Choosing the weights manually requires chess knowledge and repeated testing, and the effect of one weight may depend on the values of others.

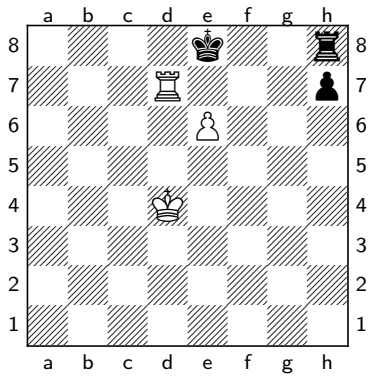


Figure 1 A position with equal material but a positional advantage for White.

Evaluation functions can be tuned or learned in several ways. Some methods rely on manual tuning and expert knowledge. Others use reinforcement learning, temporal-difference learning, expert or mentor-based learning, neural-network evaluation, or self-play. Evolutionary algorithms provide another possible approach. They search through a population of candidate solutions and use selection and genetic operators to improve them over time. In the context of a chess engine, each candidate can represent one possible set of evaluation parameters.

This thesis focuses on the automatic tuning of a chess evaluation function using evolutionary computation. The structure of the evaluation function is fixed in advance and consists of hand-engineered chess features, but its numerical parameters are evolved. The evolutionary procedure used in this thesis is based on the two-stage approach proposed by David et al. [1]. In the first stage, candidate evaluation functions are trained using positions extracted from recorded games. In the second stage, selected individuals are refined through self-play.

Thesis Goal

The aim of this thesis is to implement and evaluate a simplified evolutionary system for tuning the parameters of a classical chess evaluation function. The work studies how a fixed set of hand-engineered evaluation features can be represented in an evolvable form, how recorded games can be used to guide the initial phase of evolution, and how self-play can further modify the resulting organisms.

The implementation includes a playable chess engine with legal move generation, bounded search, and a parameterized static evaluation function. It also includes an evolutionary training pipeline with the two above-mentioned stages. The engine can be tested interactively and through the Universal Chess Interface, so it can be connected to external chess interfaces. The experiments then evaluate the behavior of the evolutionary stages and discuss the limitations of the approach.

The purpose of the work is not to create a chess engine competitive with modern top-level engines. Instead, the goal is to build a functional experimental environment in which evolutionary tuning of a chess evaluation function can be implemented, tested, and analyzed.

Thesis Structure

The thesis is divided into four chapters. *Chapter 1* introduces the necessary background on evolutionary algorithms, chess engines, and related approaches to chess engine optimization. *Chapter 2* describes the evolutionary design used in this work, including the evaluation function, genome representation, and the two-stage optimization process. *Chapter 3* focuses on the implementation of the chess engine, training pipeline, experiment infrastructure, and testing interfaces. *Chapter 4* presents the experimental setup, results, final organism, and limitations of the approach.

The conclusion summarizes the achieved results, evaluates whether the thesis goal was met, and suggests possible directions for future work.

1 Background and Related Work

1.1 Evolutionary Algorithms

Evolutionary algorithms are optimization methods inspired by biological evolution. They are especially useful when the search space is large, irregular, or difficult to describe analytically. Instead of improving a single solution directly, an evolutionary algorithm works with a population of candidate solutions. These candidates are evaluated, selected, modified, and used to form new generations. Over time, the population is expected to move toward better solutions [4].

In this thesis, evolutionary algorithms are used to tune the numerical parameters of a chess evaluation function. The evaluation function has a fixed structure, but its weights are not chosen manually. Instead, they are encoded in genomes and optimized through an evolutionary process.

1.1.1 Individuals, Genomes, and Populations

The basic unit of an evolutionary algorithm is an individual. An individual represents one candidate solution to the problem being solved. The encoded form of the individual is called the genotype or genome. The actual solution obtained from this encoding is called the phenotype.

For example, in a parameter optimization problem, the genome may be a vector of bits or numbers. After decoding, this genome becomes a concrete set of parameter values. The evolutionary algorithm operates on the genome, while the fitness function evaluates the behavior or quality of the decoded solution.

A population is a set of individuals. The population gives the algorithm several candidate solutions at the same time, rather than only one. This is important because it allows the algorithm to explore different parts of the search space. At the beginning of a run, the initial population is usually generated randomly or initialized from some prior information. The algorithm then repeatedly creates new generations from the previous one.

1.1.2 Fitness Function

The *fitness function* defines what it means for an individual to be good. It assigns a fitness value to each candidate solution. In a simple optimization problem, this value may be computed directly from a mathematical objective. In more complex problems, fitness may be obtained by simulation, testing, or comparison with reference data.

The choice of fitness function is one of the most important parts of an evolutionary algorithm. If the fitness function does not capture the desired behavior, the algorithm may optimize the wrong property. In game-playing domains, fitness can also be noisy or expensive to compute. For example, evaluating a chess-playing individual may require many positions or games. A single game result may not be reliable, because it can depend on the opening, color, search depth, or one tactical mistake.

In this thesis, fitness is used in two different ways. In the first stage, candidate evaluation functions are compared with moves from recorded games. In the second stage, individuals are evaluated through self-play. These two fitness definitions are described in detail in Chapter 2.

1.1.3 Selection

Selection determines which individuals are allowed to contribute to the next generation. Better individuals should have a higher chance of being selected, but it is usually not desirable to keep only the best candidates. If the population becomes too uniform too early, the algorithm may converge to a poor local optimum.

One common selection method is *tournament selection*. In tournament selection, a small group of individuals is chosen randomly from the population. The best individual in that group is selected as a parent. The size of the tournament controls the selection pressure. A larger tournament gives stronger individuals a higher chance of being selected, while a smaller tournament preserves diversity for longer.

Selection creates a balance between *exploitation and exploration*. Exploitation means using the best solutions found so far. Exploration means continuing to search new areas of the solution space. A good evolutionary setup must balance both.

1.1.4 Genetic Operators

New individuals are created using genetic operators. The two most common operators are crossover and mutation [5].

Crossover combines genetic material from two or more parents. In a bit-string representation, this may mean taking one part of the genome from one parent and another part from another parent. The purpose of crossover is to combine useful partial solutions that already exist in the population.

Mutation introduces random changes into a genome. In a bit-string representation, mutation may flip selected bits. Mutation can introduce new genetic material and help the algorithm explore parts of the search space that are not reachable by crossover alone.

Many evolutionary algorithms also use *elitism*. Elitism means that one or more of the best individuals are copied directly into the next generation. This prevents the best solution found so far from being lost due to random variation. However, elitism must be used carefully. Too much elitism can reduce diversity and make the population converge too quickly.

1.1.5 Genetic Algorithm

A *genetic algorithm* is a well-known type of evolutionary algorithm. It typically uses a fixed-length genome, a fitness function, selection, crossover, and mutation [5]. A simple genetic algorithm can be described as the iterative procedure shown in Algorithm 1. The algorithm starts with an initial population, evaluates its individuals, and then repeatedly applies selection and genetic operators to produce new generations.

Algorithm 1 Simple genetic algorithm

Input: population size N, number of generations G
Input: crossover probability p_c, mutation probability p_m, fitness function f
Output: best individual found

```
Initialize population P with N individuals
Evaluate all individuals in P using f
b ← best individual in P
for g ← 1 to G do
    Select parent individuals from P according to fitness
    Apply crossover with probability p_c to create offspring
    Apply mutation with probability p_m to the offspring
    Evaluate the offspring using f
    Form the next population P from the offspring, possibly using elitism
    Update b if a better individual is found
end for
return b
```

In each generation, individuals with better fitness are more likely to contribute genetic material to the next population. Over time, this can improve the population with respect to the chosen fitness function. In this thesis, the same general idea is used to evolve chess evaluation functions: each genome represents one possible set of evaluation parameters, and the fitness function measures how useful the decoded evaluator is in the current training stage.

1.1.6 Genetic Programming

Genetic programming is another form of evolutionary computation. Unlike a typical genetic algorithm, where the genome often encodes a fixed-length vector of parameters, genetic programming evolves program-like structures. Individuals may be represented as trees composed of functions and terminals, and the evolutionary process modifies or recombines these trees [6].

For example, in a game-playing task, a genetic programming individual could represent a small evaluation expression or decision rule. Internal nodes of the tree could be arithmetic operations or conditional functions, while leaves could be features such as material balance, piece mobility, king safety, passed-pawn advancement, or pawn structure. This allows the structure of the decision rule itself to change during evolution.

1.2 Game Playing and Chess Engines

1.2.1 Games

Many board games can be described as deterministic, two-player, zero-sum games with perfect information. Deterministic means that there is no randomness after a move is chosen. Perfect information means that both players can observe the complete state of the game. Zero-sum means that an advantage for one player corresponds to a disadvantage for the other. Chess belongs to this class of games,

which is why it is often used as an example in artificial intelligence and game-tree search [7].

A common way to model such games is a game tree. Each node represents a game state, and each edge represents a possible move. The players move alternately. Since their goals are opposite, one player tries to maximize the evaluation of the position, while the other tries to minimize it. This leads to the minimax principle: a move is good if it leads to a position that remains favorable even after the opponent chooses a strong reply.

The complete game tree is often too large to search exhaustively. The number of possible continuations grows quickly with search depth, because every move can lead to many possible replies. For this reason, game-playing programs normally search only part of the tree and use an evaluation method at the search boundary.

Alpha-beta pruning is a general improvement of minimax search. It avoids searching branches that cannot influence the final minimax decision. When applied correctly, alpha-beta pruning returns the same result as full minimax search, but it may evaluate far fewer positions [8]. Its effectiveness depends strongly on move ordering. If promising moves are searched first, more branches can be pruned.

1.2.2 Chess

Chess is a deterministic two-player game with perfect information, but its search space is very large. Shannon already observed that exhaustive search is infeasible for chess and estimated the number of possible games to be enormous [2]. This makes chess a useful domain for studying search, evaluation, and later also learning and optimization.

A classical search-based chess engine usually contains a board representation, a move generator, a search algorithm, and an evaluation function. The board representation stores the current position. The move generator provides legal moves or candidate moves for the search. In a full-width search, all legal moves in the expanded positions are considered. Advanced engines may use pruning, reductions, extensions, or other selective techniques, but they still need to avoid illegal moves and search only continuations that can actually occur in the game.

The search algorithm explores possible continuations from the current position. Search depth is one of the main factors influencing engine strength. A deeper search can detect longer tactical sequences, such as captures, checks, threats, or checkmating attacks. However, each additional ply increases the number of positions that may need to be searched.

For this reason, practical chess engines use many additional techniques. These may include iterative deepening, transposition tables, move ordering heuristics, quiescence search, opening books, and endgame tablebases. Deep Blue, for example, combined highly optimized search, specialized hardware, a complex evaluation function, and a database of grandmaster games [3].

Quiescence search is used because evaluating a tactically unstable position can be misleading. For example, if a search stops immediately before an obvious recapture, the static evaluation may overestimate the value of a move. Quiescence search extends the search in selected forcing lines, usually involving captures, checks, or direct threats. A forcing line is a sequence in which the opponent has only a small number of reasonable replies, so the continuation is more concrete

than in a quiet strategic position.

1.2.3 Chess Data Formats and Interfaces

Chess engines and training pipelines need textual formats for storing games, representing positions, and communicating with external tools. Two data formats and one protocol are especially relevant for this thesis: PGN, FEN, and UCI.

Portable Game Notation (PGN) is a plain-text format for storing complete chess games [9]. A PGN file usually contains tag pairs with metadata, such as the players, event, date, and result, followed by the moves of the game in algebraic notation. A small PGN example is shown below:

```
[White "Carlsen"]
[Black "Nakamura"]
[Result "1-0"]

1. e4 e5 2. Nf3 Nc6 1-0
```

Forsyth–Edwards Notation (FEN) is used to represent a single chess position in one line [10]. A FEN string contains the piece placement, side to move, castling rights, en passant target square, halfmove clock, and fullmove number. For example, the initial chess position is represented as:

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

The first part describes the board from rank 8 to rank 1. Letters represent pieces, uppercase letters are White pieces, lowercase letters are Black pieces, and digits represent consecutive empty squares. For example, the position shown in Figure 1 can be represented by the following FEN string:

```
5k1r/3R3p/4P3/8/3K4/8/8/8 w - - 0 1
```

The Universal Chess Interface (UCI) is not a game notation format, but a communication protocol between a chess engine and a graphical user interface [11]. It allows a GUI to send commands such as position setup and search requests to the engine. UCI moves are written in coordinate form, for example `e2e4` or `e7e8q` for a promotion.

These formats and the UCI protocol are used in different parts of this thesis. PGN game records are used as a source of training data. Positions extracted from these games are stored as FEN strings, and the corresponding reference moves are stored in UCI notation. The implemented engine also supports UCI mode, so it can be tested in external chess interfaces such as Arena.

1.2.4 Evaluation Functions

Since a practical chess engine cannot search the complete game tree, it needs a way to estimate non-terminal positions at the search boundary. In classical engines, this is done by a static evaluation function. The evaluation function assigns a numerical score to a position. Positive values usually represent an advantage for

White, negative values represent an advantage for Black, and values close to zero represent an approximately equal position.

A minimalistic evaluation function could be based only on material. Such an evaluator assigns values to pieces and computes the difference between White's and Black's material. Material is important, but it is not sufficient. Two positions with the same material balance may differ in king safety, pawn structure, piece activity, passed pawns, open files, or control of important squares.

For this reason, classical evaluation functions often combine multiple features. Each feature describes some property of the position, and its weight determines how much it contributes to the final score. For example, the evaluator may add a bonus for a passed pawn, subtract a penalty for an isolated pawn, or add a bonus for a rook on an open file. This kind of parameterized evaluation function is the main object optimized in this thesis.

The distinction between hand-engineered and learned evaluation is important. In a hand-engineered evaluation function, a domain expert defines the features, while the weights may be chosen manually or tuned automatically. Modern engines may also use neural-network evaluation. For example, Stockfish introduced efficiently updatable neural network (NNUE) evaluation, where a neural network computes the position value used by alpha-beta search, while earlier classical evaluation was based on handcrafted chess concepts [12]. The engine in this thesis follows the classical approach: the features are hand-engineered, but their numerical weights are optimized by an evolutionary algorithm.

1.3 Chess Engine Optimization

1.3.1 Manual and Parameter-Based Tuning

The strength of a classical chess engine depends heavily on both search and evaluation. Traditionally, many evaluation functions were designed and tuned manually. The engine author or domain expert selected chess features and adjusted their weights using chess knowledge, testing, and experience. This approach can work well, but it becomes difficult when the number of parameters grows. The effect of one parameter may depend on many others, and a change that improves one type of position may make another type worse.

A natural automatic alternative is to keep the evaluation features fixed and tune their weights algorithmically. Tunstall-Pedoe investigated the use of genetic algorithms for optimizing the weights of a static chess evaluation function [13]. Kendall and Whitwell also used an evolutionary approach for tuning a chess evaluation function, with fitness based on game outcomes and population dynamics [14]. These works are closely related to this thesis because they treat the evaluation function as a parameterized object that can be optimized.

The main difficulty of game-based tuning is the cost and noise of evaluation. If fitness is based on games, then many games may be needed to distinguish a stronger individual from a weaker one reliably. A single game can be influenced by the opening, color, search depth, or one tactical mistake. This is why chess-engine optimization often requires careful experimental design.

1.3.2 Temporal-Difference Learning

Another approach to learning an evaluation function is *temporal-difference learning*. Unlike a genetic algorithm, temporal-difference learning does not evaluate a whole population of candidate genomes. Instead, it usually updates one evaluation function gradually during play or from recorded game trajectories. The main idea is that the evaluation of an earlier position should become more consistent with the evaluation of later positions.

For example, suppose that an engine evaluates a position as approximately equal, but after several moves the resulting position is evaluated as clearly better for White. Temporal-difference learning can use this difference as a training signal and adjust the earlier evaluation so that it better predicts the later development of the game. The method therefore does not need to wait only for the final result of the game. It can learn from the differences between successive predictions.

Baxter, Tridgell, and Weaver introduced TDLeaf(λ), a temporal-difference method designed for learning an evaluation function in combination with minimax search [15]. In ordinary temporal-difference learning, the update may compare consecutive states directly. In chess, however, the engine does not choose moves only from the current board position in isolation; it searches a game tree and evaluates positions at the leaves. TDLeaf(λ) adapts the temporal-difference idea to this setting by using the leaf positions selected by search.

Compared with the population-based approach used in this thesis, temporal-difference learning is an incremental learning method: it changes one evaluation function step by step instead of selecting among many candidate genomes.

1.3.3 Self-Learning Evolutionary Players

Evolutionary methods have also been applied to broader chess-playing behavior. Fogel et al. presented a self-learning evolutionary chess program that improved through self-play rather than by copying a human expert or a fixed chess engine [16]. These approaches show that evolutionary learning can be used without a direct teacher.

However, pure self-play from weak initial individuals can be difficult. If the starting population plays very poorly, the early fitness signal may not guide the search toward strong chess behavior. Individuals may become better relative to each other without becoming strong in an absolute sense. This motivates hybrid approaches in which an initial supervised or mentor-guided phase is followed by self-play.

1.3.4 Genetic Programming for Chess

Genetic programming has also been used in chess-related tasks. The main difference from the genetic-algorithm approach used in this thesis is what is being evolved. In a genetic algorithm for parameter tuning, the structure of the solution is fixed in advance and evolution changes only the parameter values. In this thesis, for example, the set of evaluation features is fixed, and the genome determines their numerical weights.

In genetic programming, the evolved individual can be a program, expression, or decision rule. For a chess evaluation task, one individual could represent an

expression that combines material balance, piece mobility, king safety, passed-pawn advancement, or pawn-structure terms. Another individual could have a different structure, for example one that gives a bonus for a passed pawn only when it is sufficiently advanced or supported by the king. The evolutionary process can therefore modify not only the numerical values, but also the form of the rule itself.

Hauptman and Sipper applied genetic programming to evolve players for selected chess endgames [17]. Endgames are a natural setting for this type of experiment because the number of pieces is smaller and the task can be more clearly constrained than in full chess. An evolved individual can be tested by how well it plays or solves positions from the selected endgame domain.

The advantage of genetic programming is its flexibility. It can search for structures that were not explicitly specified by the engine author. The disadvantage is that this also creates a much larger and more difficult search space. The resulting expressions may become complex, hard to interpret, or too specialized to the training positions. For this thesis, a fixed evaluation structure is more suitable because it keeps the implementation and experiments manageable while still allowing automatic tuning of the evaluation parameters.

1.3.5 Expert-Driven Learning

A particularly relevant line of work was developed by David and co-authors. One approach used genetic algorithms to simulate evaluation functions by learning from an expert or mentor program [18]. In this setting, the target program provides information that guides the evolutionary process. The advantage is that a strong mentor can give a useful training signal. The disadvantage is that the method depends on access to such an expert.

Another approach removed this dependency by learning from games played by human grandmasters. David-Tabibi et al. evolved evaluation functions first by imitating moves from recorded grandmaster games and then by improving the resulting organisms through coevolution [19]. The later journal version by David et al. extended this line of work and also considered the evolution of search parameters [1].

The method used in this thesis follows the same general direction, but with a narrower scope. It focuses only on tuning the parameters of a fixed evaluation function. It does not aim to evolve a tournament-level engine, and it does not optimize the full search mechanism. The implementation also uses a smaller experimental setup, limited search budgets, and a simplified training pipeline. These simplifications make the approach feasible for this thesis while preserving the central idea: recorded games provide an initial training signal, and self-play is then used to refine the evolved evaluation parameters.

2 Evolutionary Design

This chapter describes the design of the evolutionary approach used in the thesis. The method follows the general two-stage idea of David et al. [1]. The chess engine uses a fixed evaluation function structure composed of hand-engineered features. Evolution does not change which features are present; instead, it searches for suitable numerical weights for those features.

The design has four main parts. First, a parameterized chess evaluation function defines how positions are scored. Second, a genome representation encodes the numerical parameters of this function. Third, Stage 1 uses positions extracted from recorded games to guide the population toward reasonable evaluation parameters. Fourth, Stage 2 refines selected individuals through self-play.

2.1 Evaluation Function Design

The evaluation function is the object optimized by the evolutionary process. It receives a chess position and returns a numerical score in centipawns. The structure of the evaluation function is fixed in advance, while the numerical weights of its features are decoded from the genome.

2.1.1 Score Convention

The evaluator uses a White-centered score convention. A positive score means that the position is considered better for White, while a negative score means that it is considered better for Black. A score close to zero represents an approximately equal position. This convention is useful because it makes the evaluation independent of the side to move. The search algorithm can later convert the score to the perspective of the player whose move is being selected.

The unit used for most terms is the centipawn. A pawn has the fixed value of 100 centipawns and serves as the reference point for the other material and positional terms. Other piece values and feature weights are evolved. For example, the genome determines the values of knights, bishops, rooks and queens, but the pawn value remains fixed.

The total evaluation can be viewed as a weighted sum of features:

$$E(s) = \sum_i w_i f_i(s),$$

where s is the evaluated position, $f_i(s)$ is a feature extracted from the position, and w_i is the corresponding weight. In the implementation, some terms are computed as bonuses and others as penalties, but the principle is the same: each feature contributes to the final centipawn score.

2.1.2 Hand-Engineered Features

The evaluation function contains several groups of hand-engineered features. These features were selected to represent common ideas used in traditional chess evaluation, such as material balance, pawn structure, king safety and piece activity.

The purpose of evolution is not to discover these concepts from scratch, but to tune their relative importance.

Feature group	Included terms
Material	Values of knights, bishops, rooks and queens. The pawn value is fixed at 100 centipawns.
Bishop pair and tempo	Bonus for having two bishops and bonus for the side to move.
Pawn structure	Doubled pawns, isolated pawns, backward pawns, weak squares, passed pawns, protected passed pawns, connected passed pawns, and enemy king distance from the promotion square of a passed pawn.
King safety	Pawn shield in front of the king, open or semi-open file near the king, and attacking pieces close to the king.
Mobility	Mobility of bishops, rooks and queens, measured by the number of reachable squares.
Rook features	Rooks on open or semi-open files, rooks on the seventh rank, connected rooks, rooks behind passed pawns, rook pressure on the enemy king, and attacks on weak pawns.
Knight features	Knight centrality and knight outposts.

Table 2.1 Main feature groups used by the evaluation function.

Most feature groups are evaluated symmetrically. The evaluator computes the contribution for White and for Black and returns the difference. For example, a passed pawn bonus is added for White's passed pawns and subtracted for Black's passed pawns. This keeps the evaluation function consistent with the White-centered score convention.

The feature set is hand-engineered and interpretable. Each term corresponds to a recognizable chess concept. Evolution does not remove these feature detectors or create new ones. Instead, it changes how much each detected feature contributes to the final score. One organism may assign a high value to passed pawns, while another may value king safety or rook activity more strongly.

2.2 Genome Representation

Each individual in the population represents one candidate evaluation function. The individual does not store the evaluation parameters directly as a list of integers. Instead, it stores a bit-string genome that is decoded into parameters using a schema.

2.2.1 Schema-Based Bit Encoding

The genome is divided into genes. Each gene corresponds to one named evaluation parameter and has a fixed bit width and numeric range. The schema therefore defines the layout of the genome: which parameters are present, how

many bits are assigned to each of them, and what minimum and maximum values they may decode to.

Conceptually, a gene specification has the form

$$(\text{name}, \text{bit width}, \text{minimum}, \text{maximum}).$$

For example, one gene may encode the knight value, another the bishop pair bonus, and another the passed pawn bonus. The full genome is created by concatenating all gene bit strings in the order defined by the schema.

Using a schema-based representation has two advantages. First, the evolutionary algorithm can operate on a simple bit string without knowing the meaning of each parameter. Second, the set of evolved parameters can be changed by editing the schema, while the genetic operators can remain the same.

2.2.2 Decoding Genomes into Parameters

Before an individual can be evaluated, its genome must be decoded into concrete evaluation parameters. Each gene is decoded separately. The bit sequence of the gene is interpreted using Gray coding and then mapped to the numeric range specified by the schema. The decoded value is rounded to an integer, because the evaluation parameters are stored as integer centipawn values or integer multipliers.

Gray coding is used because neighboring encoded integer values differ by only one bit. This can make mutation smoother than ordinary binary encoding, where a small numeric change may require flipping several bits. After decoding, the resulting parameter object is passed to the evaluation function.

The decoder maps gene names to fields of the evaluation parameter structure. If a gene named for a supported feature is present in the schema, its decoded value replaces the corresponding default parameter. Parameters that are not present in the schema keep their default values. This makes the genome representation flexible: experiments can evolve only a subset of parameters or a larger schema without changing the evaluator itself.

2.2.3 Genetic Operators

The same genetic operators are used in both training stages. Parent selection is performed by tournament selection. A small number of individuals is sampled from the population, and the strongest individual in this group is selected as a parent. The tournament size controls the selection pressure.

Crossover creates an offspring genome from two parent genomes. The implementation supports uniform crossover at the bit level and uniform crossover at the gene level. In the gene-level variant, each whole gene is copied from one of the two parents. This is useful because it preserves the internal bit structure of a decoded parameter. In the bit-level variant, each bit can be inherited independently.

Mutation is implemented as bit-flip mutation. When mutation is applied to a genome, each bit has a configurable probability of being flipped. This allows the algorithm to introduce new genetic material and explore parameter values that may not be present in the current population.

Elitism can also be used. With elitism, one or more of the best individuals from the current generation are copied into the next generation before new offspring

are created. This prevents the best genome found in a generation from being lost immediately due to crossover or mutation.

2.3 Stage 1: Evolution from Recorded Games

The first evolutionary stage uses recorded chess games as a training signal. Its purpose is to create evaluation functions that prefer moves similar to those played by strong human players in real games. This gives the population a more reasonable starting point before Stage 2.

2.3.1 Training Positions

The training data consists of pairs

$$(\text{FEN position}, \text{UCI move}),$$

where the FEN describes the position before a move and the UCI move is the move selected from the recorded game. The data can either be loaded directly from a TSV file or generated from PGN game records.

When PGN games are used, the dataset builder follows the same general idea of learning from decisive games used by David et al. [1]. Only decisive games are used by default. Drawn games and games with unknown results are skipped. From each eligible game, one position is selected where the side to move is the eventual winner of the game. The move played in the game then becomes the reference move for that training sample.

This produces one training sample per selected game. The position is stored as a FEN string before the move, and the reference move is stored in UCI notation. After enough samples are collected, the data are shuffled and split into training and test sets. The training set is used to compute fitness during evolution, while the test set is used to report how well the best individual generalizes to unseen samples.

2.3.2 Fitness Computation

For each training sample, the candidate evaluation function is used to choose a move from the given position. Stage 1 uses a one-ply search. This means that the engine generates legal moves, makes each candidate move, evaluates the resulting position, and selects the move with the best score.

The selected move is then compared with the reference move from the recorded game. If the moves match exactly, the sample is counted as correct. The number of correct predictions over the training batch is the basic measure of performance.

The fitness value is computed as the square of the number of correct predictions:

$$F = c^2,$$

where c is the number of correctly predicted moves. Squaring the number of correct moves increases the difference between weak and strong individuals. For example, an individual that predicts twice as many moves correctly receives more than twice the fitness.

This fitness function is simple and easy to compute, but it has limitations. A recorded move is not always the only good move in a position. Chess positions may contain several reasonable alternatives, and an evaluator can choose a good move that does not match the game record exactly. The reference move is also a human move. Although the games are assumed to be high-quality, the move is not guaranteed to be objectively best. Therefore, Stage 1 should not be interpreted as a perfect measure of chess strength. Its main purpose is to guide the initial population toward parameters that produce plausible chess moves.

2.3.3 Train-Test Split

The train-test split is used to detect whether the evolved parameters only fit the training positions or whether they also perform reasonably on unseen examples. During each generation, individuals are evaluated on the fixed training set. The best individual of the generation is then also evaluated on the test set, and its test accuracy is recorded.

The test set is not used for selection. It is only used for reporting. This distinction matters because selecting directly on the test set would make the test accuracy less useful. In the experiments, the difference between training and test behavior is later used to discuss whether Stage 1 produces general improvement or mainly adapts to the selected training samples.

2.4 Stage 2: Self-Play Evolution

The second stage evaluates individuals by letting them play chess games against each other. This changes the training signal from agreement with recorded moves to direct competitive performance. The goal is to refine the evaluation parameters in the environment in which the final organism is expected to be used.

2.4.1 Initial Population

Stage 2 can start from a random population, but the intended pipeline uses good individuals from Stage 1 as seeds. In this setup, selected Stage 1 checkpoints are inserted into the initial Stage 2 population. If fewer Stage 1 genomes are available than the Stage 2 population size, the remaining individuals are initialized randomly.

This design reflects the purpose of the two-stage method. Stage 1 provides an initial population shaped by recorded games, while Stage 2 performs the more expensive self-play evaluation. This follows the motivation of David et al. [1]: using game records first can give evolution a useful starting point before the computationally heavier coevolutionary stage. Starting Stage 2 only from random genomes would require the costly game-based fitness function to improve individuals that may initially play very weakly.

2.4.2 Self-Play Fitness

In Stage 2, individuals are evaluated using a round-robin tournament. Each pair of individuals plays a small match. The games in a match alternate colors,

so that one individual does not always receive the advantage or disadvantage of playing the same color.

The result is converted into points from the perspective of each individual. A win is worth two points, a draw is worth one point, and a loss is worth zero points. After all matches are played, the total points of each individual are normalized by the maximum number of points it could have obtained. The resulting relative fitness is therefore in the interval from 0 to 1.

This fitness value is relative. It does not measure absolute chess strength; it measures performance against the opponents used in that generation. An individual can receive a high fitness because it performs well against the current population, even if it would not be strong against external opponents. This is one of the main difficulties of self-play evolution and is discussed again in the experimental chapter.

2.4.3 Search Budget

The strength and cost of self-play evaluation depend on the search budget used during games. A larger search budget usually gives more reliable games, but it also makes evolution much slower. Since Stage 2 requires many games, this trade-off is important.

The implementation supports several search modes. A game can be played with a fixed search depth, with a node limit, with a fixed amount of time per move, or with a game clock. In time-based modes, the search uses iterative deepening and returns the best move found within the time budget. A maximum search depth can still be used as a safety limit.

The design therefore does not assume only depth-limited play. Search can be bounded by depth, nodes, or time depending on the experiment configuration. In Chapter 4, the concrete search budgets used in the experiments are reported together with the results.

The search also includes a simple draw-handling mechanism. Draws are normally scored as neutral results, but during search the engine can be made less willing to voluntarily choose a draw when its static evaluation says that it is clearly better. This was added to reduce cases where a promising position is converted into an unnecessary draw.

2.4.4 Hall of Fame

Self-play evolution can suffer from short-term and relative fitness. A new individual is mainly compared with the current population. If the whole population drifts in a weak direction, an individual can still look good relative to its current opponents.

To reduce this problem, Stage 2 can use a Hall of Fame mechanism. The Hall of Fame stores strong genomes found in previous generations. When the mechanism is enabled, current individuals play additional games against selected Hall of Fame opponents. This adds evaluation pressure from earlier successful organisms and makes it harder for the population to forget how to beat previous strategies.

The Hall of Fame has a fixed maximum size. When a new best individual is added, older entries may be removed to keep the list bounded. The mechanism therefore provides a limited memory of earlier generations without making evaluation too expensive.

2.4.5 Opening Positions

The starting position of self-play games can strongly affect the results. If every game starts from the standard initial chess position, many games may repeat similar opening mistakes, especially because the evolved engine does not use a full opening book for move selection.

For this reason, Stage 2 can use a file of starting positions in FEN format. Before each game, one starting position is selected randomly from the list. The file may also contain the standard starting position. This is not a move-by-move opening book; it is a collection of positions from which games may begin.

Using starting FENs has two advantages. First, it increases variety in self-play games. Second, it can reduce the influence of very early opening play on the evaluation of organisms. The organisms are still tested by playing complete games from the selected positions, but the tournament is less dependent on repeatedly reaching the same opening positions.

During Stage 2, the best genome of each generation is stored as a checkpoint. These checkpoints can later be inspected, tested further, or used as candidate final organisms in the experimental evaluation. The following chapter describes how this design is implemented in the program.

3 Implementation

This chapter describes how the evolutionary design from Chapter 2 was implemented. The implementation is written in `C#` on .NET 8 and is split into a core library, a console application, and a test project. The core library contains the chess engine, evaluation function, genome handling, data preparation, training loops, checkpointing, and metric writing. The console application provides command-line entry points for running the two evolutionary stages and for testing evolved organisms interactively or through a chess graphical user interface. The test project contains unit and smoke tests for the most important parts of the system.

The chapter focuses on the implementation structure and the responsibilities of the main components. The experimental settings and numerical results are discussed separately in Chapter 4.

3.1 Project Structure

The solution is divided into three main projects:

`ChessGP.Core` The core library. It contains the chess engine abstraction, search implementation, parameterized evaluator, genome decoding, genetic algorithm operators, PGN ingestion, training drivers, checkpointing, and metric writing.

`ChessGP.App` The console application. It loads the configuration file and starts the requested mode, such as Stage 1 training, Stage 2 training, interactive play, or UCI mode.

`ChessGP.Tests` The test project. It contains automated tests for the board adapter, search, evaluator features, genome encoding, genetic operators, data ingestion, persistence, and training drivers.

This separation keeps the implementation modular. The training logic and chess engine do not depend on the console interface, and the tests can instantiate core classes directly. The application project is therefore mostly responsible for command-line parsing and configuration loading, while the core project contains the actual thesis functionality.

The core project is further organized into several namespaces. The `Engine` namespace contains the board adapter, move representation, evaluator interface, and search engine. The `Eval` namespace contains the decoded evaluation parameters, genome-to-parameter decoder, and parametric evaluator. The `Genome` namespace contains the schema representation and Gray-code conversion. The `GA` namespace contains genetic algorithm operators. The `Data` namespace contains PGN and training-sample processing. The `Training` namespace contains Stage 1, Stage 2, match running, game running, and opening-position loading. The `Persistence` namespace contains checkpoint and metric output.

3.2 Configuration and Application Entry Points

The program is controlled by a JSON configuration file. The configuration contains sections for data input, Stage 1 parameters, Stage 2 parameters, genetic algorithm settings, genome schema, random seed, and output paths. This makes experiments reproducible, because the important settings are stored outside the source code.

The configuration is loaded into strongly typed option records. The main configuration object contains the following groups:

`Data` Input path, input format, filters, and labeling options.

`Stage1` Population size, crossover and mutation rates, number of generations, batch size, and search ply.

`Stage2` Population size, game settings, search budget, Hall of Fame settings, opening positions, and draw-handling parameters.

`GA` Selection type, tournament size, bit-flip rate, crossover type, and elitism.

`Genome` Schema identifier and gene definitions.

`Rng` Random seed.

`Output` Checkpoint directory and metrics file.

Paths from the configuration are resolved relative to the directory containing the configuration file. This is useful when running experiments in different environments, because a copied configuration can refer to nearby data, checkpoint, and metric files without depending on the original working directory.

The console application supports several modes. The `run-stage1` command starts the first training stage. The `run-stage2` command starts self-play evolution. The `play` command loads an evolved checkpoint and allows a human to play against it in the terminal. The `uci` command loads a checkpoint and runs the engine through the UCI protocol, so that it can be tested in external chess interfaces.

This design allows the same codebase to support training and testing. The training modes are used for experiments, while the play and UCI modes are useful for inspecting the behavior of evolved organisms qualitatively.

3.3 Chess Engine Core

3.3.1 Board Adapter

The chess engine uses a board adapter interface rather than exposing a particular chess library directly. The main implementation is `ChessDotNetBoardAdapter`, which wraps the `ChessDotNet` library. The adapter provides methods for loading the starting position, loading a FEN position, generating legal moves, making and unmaking moves, checking terminal positions, detecting threefold repetition, reading the halfmove clock, and converting the current position back to FEN.

The adapter uses UCI moves internally. A move stores the source square, target square, and optional promotion piece. This is the same notation used by the training samples and by the interactive and UCI interfaces.

Move making and unmaking are implemented using FEN snapshots. Before a move is made, the adapter stores the current FEN as an unmake token. To unmake the move, the adapter reloads the saved FEN. This approach is not the fastest possible implementation, but it is simple and reliable for an experimental engine. Since the thesis focuses on evaluation-function tuning rather than high-performance move generation, this trade-off is acceptable.

The adapter also tracks repeated positions. It stores a repetition key derived from the FEN fields that identify the board position, side to move, castling rights, and en passant square. The halfmove and fullmove counters are not part of the repetition key. When a position appears three times, the adapter reports a threefold repetition draw. The game runner also handles the fifty-move rule using the halfmove clock.

3.3.2 Search

The search logic is implemented in `EngineCore`. The class supports two main search modes. Stage 1 uses `SelectBestMove1Ply`, which evaluates all legal moves after one ply and selects the move with the best resulting score. Stage 2 and external testing use `SearchAlphaBeta`, which implements a negamax version of alpha-beta search.

The evaluator returns a White-centered score, but search decisions must be made from the root player's perspective. For this reason, the search converts evaluation values according to the side to move at the root. Positive search scores mean that the move is good for the player whose move is being selected.

The alpha-beta search supports several limits through a `SearchLimits` object. A search may be bounded by maximum depth, node cap, time per move, or a combination of these constraints. When a time limit is used, the engine performs iterative deepening. It searches depth one first, then depth two, and so on until the depth limit or time budget is reached. The engine keeps a legal fallback move, so even a very small time budget still returns a move.

Terminal positions are handled explicitly. Checkmate is represented using a large mate score adjusted by ply, so that faster mates are preferred and slower losses are delayed. Draws are normally neutral. However, the implementation also contains a draw-contempt mechanism. When the static evaluation indicates that the engine is clearly ahead, voluntary draws can be penalized. This was added because self-play and practical testing sometimes produced cases where the engine accepted a draw despite having a clearly better position.

The search implementation is intentionally limited. It does not include advanced techniques such as transposition tables, quiescence search, or sophisticated move ordering. These techniques are important in strong chess engines, but the purpose of this implementation is to provide a working experimental environment for evolutionary tuning.

3.4 Evaluation and Genome Decoding

3.4.1 Evaluation Parameters

The evaluation parameters are represented by the `DecodedParams` record. It stores the numerical weights used by the parametric evaluator. The pawn value is fixed at 100 centipawns, while the other material values and positional terms are represented as configurable integer parameters.

The record also defines reasonable default values. These defaults are used as a baseline when a genome does not contain all possible genes. This makes the system flexible: an experiment may evolve only a subset of parameters, while the remaining parameters keep their default values.

3.4.2 Parametric Evaluator

The actual evaluation function is implemented by `ParametricEvaluator`. It reads the current position from FEN, parses the board placement, counts pieces, identifies kings and pawns, and computes the feature groups described in Chapter 2. The final value is returned as a White-centered centipawn score.

The evaluator computes material balance using the fixed pawn value and the decoded values for knights, bishops, rooks, and queens. It also computes bonuses and penalties for bishop pair, tempo, pawn structure, weak squares, king safety, piece mobility, rook activity, and knight features.

All terms are evaluated symmetrically. For example, the evaluator computes pawn-structure bonuses and penalties for White and for Black, then subtracts the Black value from the White value. This keeps the sign convention consistent throughout the evaluator.

The evaluator is deliberately separated from the search. It implements the `IEvaluator` interface, so the search engine only needs a method that maps a board position to an integer score. This separation allows the evaluator to be tested independently and different evaluators to be constructed from different genomes.

3.4.3 Genome Decoding

The genome is represented as a string of bits. The `GeneSchema` class describes how the bit string is divided into named genes. Each gene has a name, bit width, minimum value, and maximum value. The schema also computes the total number of bits and the offset of each gene inside the genome.

The `GrayCodec` class handles Gray-code conversion. Each gene is decoded from Gray code into an integer value and then mapped into the numeric range defined by the schema. The decoded value is rounded to an integer before being applied to the evaluation parameters.

The mapping from gene names to evaluation parameters is implemented in `GenomeToParamsDecoder`. The decoder starts from the default parameter set, then applies all genes present in the schema. If a gene name is recognized, its decoded value replaces the corresponding field in `DecodedParams`. Unknown gene names are ignored. This behavior allows older or smaller schemas to remain usable even if the evaluator supports more parameters.

3.5 Data Preparation

Stage 1 requires labeled training samples in the form of a FEN position and a reference UCI move. These samples can be loaded directly from TSV files or generated from PGN game records.

PGN data are processed by the `PgnIngestor`. The ingestor reads games, skips draws and games with unknown results, tokenizes the move text, and selects one position from each eligible game. The selected position must be one where the side to move is the eventual winner of the game. The move played in the game becomes the reference move.

To generate the sample, the ingestor replays the game up to the selected ply and stores the FEN before the move. It then plays the selected move and stores the FEN after the move. Since the game record is in SAN notation, the implementation resolves the corresponding UCI move by comparing the resulting FEN with the positions obtained by trying all legal UCI moves from the original FEN. If exactly one legal move produces the target position, that move is used as the label.

The generated samples are shuffled and split into training and test files. The files are written as tab-separated lines, with the FEN in the first column and the UCI move in the second column. If the configuration already points to existing TSV files, the system reuses them instead of rebuilding the dataset. For TSV input, the test file is derived from the training filename, for example by replacing `.train.` with `.test.`.

This preprocessing step is separated from the actual evolutionary loop. During Stage 1, the trainer loads a fixed set of samples and evaluates all individuals in a generation on the same data.

3.6 Training Implementation

3.6.1 Stage 1 Driver

Stage 1 is implemented by `Stage1Driver` and `Stage1Trainer`. The driver controls the full evolutionary loop, while the trainer evaluates one genome on a batch of training samples.

At the beginning of a run, the driver loads the configuration, resolves paths, builds the genome schema, prepares or loads the train-test datasets, and creates an initial random population. If a compatible Stage 1 checkpoint already exists, the latest saved genome can be injected into the initial population as a warm start.

Each generation follows the same structure. First, every individual in the population is evaluated on the fixed training set. The trainer decodes the genome, constructs a `ParametricEvaluator`, loads each training FEN, runs one-ply move selection, and compares the selected move with the reference UCI move. The fitness is computed as the square of the number of correct predictions.

After all individuals are evaluated, the driver records training statistics such as best fitness, mean fitness, median fitness, number of evaluated nodes, elapsed time, and the checkpoint path of the best genome. The best genome of the generation

is saved as a checkpoint. The same best genome is also evaluated on the test set, and the test accuracy is written to a separate test metrics file.

Finally, the genetic algorithm operators create the next generation. The evaluated population is passed to the GA engine, which applies elitism, tournament selection, crossover, and mutation according to the configuration. The resulting population has zero fitness until it is evaluated in the next generation.

3.6.2 Stage 2 Driver

Stage 2 is implemented by `Stage2Driver`, `MatchRunner`, and `GameRunner`. The driver controls generations and population updates. The match runner plays small matches between two evaluators. The game runner plays a single game from a selected starting position until a terminal result, draw condition, or time loss is reached.

The initial Stage 2 population is created randomly, but it can be seeded from Stage 1 checkpoints. The implementation loads the latest compatible Stage 1 checkpoint from each Stage 1 run and inserts these genomes into the initial Stage 2 population. If a compatible Stage 2 checkpoint already exists, it can also be used as a warm start.

In each generation, Stage 2 evaluates individuals through a round-robin tournament. Every pair of individuals plays a match. Colors alternate inside each match, so that one organism does not always play the same side. A win gives two points, a draw gives one point, and a loss gives zero points. After all matches are completed, each individual's score is normalized to a relative fitness value in the interval from 0 to 1.

The implementation also supports Hall of Fame matches. If the Hall of Fame is enabled and contains stored genomes from previous generations, current individuals play additional matches against selected Hall of Fame opponents. These games contribute to the individual's points. After each generation, the current best genome is inserted into the Hall of Fame, and the list is truncated to the configured maximum size.

Like Stage 1, Stage 2 saves the best genome of each generation as a checkpoint and writes generation metrics to a CSV file. In addition to fitness statistics, Stage 2 records search-related information such as total evaluated nodes, average completed search depth, average search time, and time losses.

3.6.3 Game and Match Execution

The `GameRunner` plays games using the same `EngineCore` search used by the external testing modes. It supports several search modes: fixed depth, node cap, fixed time per move, and game-clock-based time control. In the game-clock mode, the remaining clock time is divided by an estimated number of moves to go, with an additional part of the increment included in the move budget. A move overhead is subtracted as a safety margin.

The game runner also handles draw conditions. It checks terminal positions, threefold repetition, and the fifty-move rule through the halfmove clock. In addition, it contains a maximum-ply safeguard: if no terminal result or draw

condition is reached after the limit, the game is treated as a draw. In time-based modes, a player can also lose on time.

The `MatchRunner` converts game results to points from the perspective of the first evaluator in the pair. This makes the Stage 2 driver independent of colors: it only receives wins, draws, losses, and accumulated search statistics for each match.

3.6.4 Opening Positions

Stage 2 games can start either from the standard initial position or from a list of FEN positions. The `OpeningBook` class loads a text file where each non-empty line is either a full FEN position or the keyword `startpos`. Comment lines are ignored.

The term opening book is used here in a limited sense. The file does not contain move sequences or book moves. It only provides starting positions for self-play games. During a match, the game runner selects one of these FEN positions randomly and starts the game from it. This increases the variety of positions encountered during self-play and reduces dependence on repeatedly playing from the same initial position.

3.7 Checkpointing and Metrics

The training stages produce two types of persistent output: checkpoints and metrics. Checkpoints store the best genome of a generation. Metrics store numerical information about the progress of a run.

A checkpoint is written as a JSON file containing the run identifier, stage name, generation number, schema identifier, genome bit string, and timestamp. The schema identifier is used when loading checkpoints, because a genome is valid only together with the schema that defines how its bits are decoded. When an evolved organism is later used for interactive play or UCI testing, the program checks that the checkpoint schema matches the current configuration schema.

Metrics are written as CSV files. Each generation produces one row containing the run identifier, stage, generation, best fitness, mean fitness, median fitness, evaluated nodes, seed, elapsed time, and checkpoint path. Stage 2 adds search-specific fields such as average completed depth, average search time, and number of time losses. Stage 1 also writes a separate test metrics file for the best genome's performance on the test set.

Every training run receives a timestamp-based run identifier. This allows multiple independent runs to write checkpoints and metrics into the same output directory without overwriting each other.

3.8 Testing and Validation

The project contains an automated test suite based on xUnit. The tests are not intended to prove chess strength, but they help verify that the implementation behaves consistently and that important components do not break during development.

The board adapter tests check FEN loading, move generation, move making and unmaking, promotion handling, and terminal states. Additional tests cover threefold repetition. Search tests check basic move selection, alpha-beta behavior, node limits, time-limited search, and draw handling.

Evaluator tests cover individual feature groups, including material, pawn structure, weak squares, king safety, rook pressure, knight features, and mobility-related behavior. These tests are needed because many evaluation features are subtle and easy to implement incorrectly.

Genome and genetic algorithm tests cover schema construction, Gray-code conversion, crossover, mutation, and elitism. Data tests cover PGN ingestion and training-sample TSV loading. Persistence tests check checkpoint saving and loading. Training tests include smoke tests for Stage 1 and Stage 2 drivers and a determinism test for Stage 2.

The test suite therefore supports the implementation in two ways. First, it verifies individual components such as the evaluator, genome decoder, and board adapter. Second, it checks that the complete training drivers can run on small inputs without crashing. This was especially useful because the final experiments are computationally expensive and cannot be used as quick correctness tests during development.

3.9 Interactive and UCI Testing

The implementation also provides two ways to test the evolved engine outside the training pipeline. The first is an interactive console mode intended for simple manual play. It allows a selected checkpoint to be loaded and used in a playable game against the engine. This mode is useful for quick manual inspection of an evolved organism.

The second is an UCI mode. As described in Section 1.2.3, UCI allows the engine to communicate with external chess interfaces. In this mode, the program behaves as a simple chess engine that can be connected to a graphical interface supporting the UCI protocol. The UCI command loads a checkpoint, checks schema compatibility, constructs the evaluator, and responds to standard commands such as `uci`, `isready`, `position`, and `go`. The implementation also exposes configurable options such as search depth, node cap, maximum search depth, move overhead, estimated moves to go, and draw-contempt parameters.

These testing modes are useful for manual inspection because numerical fitness values alone do not fully describe how an evolved organism actually plays. Playing against the engine or observing it in a graphical interface can reveal practical weaknesses, such as poor opening play, tactical blindness at shallow depth, or repeated draw choices.

4 Experiments and Results

This chapter presents the experimental evaluation of the two-stage evolutionary method described in Chapter 2. Stage 1 is evaluated through move-prediction accuracy on positions extracted from recorded games, while Stage 2 is evaluated through self-play runs with different search budgets. The final part of the chapter describes how the final organism was selected from the evolved checkpoints.

The purpose of the experiments is not to compare the engine with modern chess engines. The implementation is intentionally limited and serves as an experimental environment for studying evolutionary tuning of a classical evaluation function. The main questions are how much training signal is obtained from the Stage 1 move-prediction task, how the Stage 2 self-play process behaves under different search budgets, and which evolved organism should be selected as the final result of the thesis.

4.1 Experimental Setup

4.1.1 Stage 1 Setup

Stage 1 was evaluated using ten independent evolutionary runs. All runs used the same training method and genome representation, but different random seeds. The goal was to observe whether the supervised stage behaves consistently and whether it produces organisms that can be used as starting points for Stage 2.

The training data consisted of positions extracted from recorded games. Each sample contained a position and a reference move played in the game. The dataset used in this thesis consisted mostly of fast-paced games involving very strong grandmasters. This differs from the reference work of David et al. [1], which used classical games of grandmasters rated above 2600 Elo. The difference in data source is important when interpreting the results, because moves played under fast time controls can be more influenced by time pressure and may be more volatile than moves from classical games.

Both the training set and the test set contained 4000 samples. During training, each organism was evaluated on the training set using one-ply move selection. The selected move was compared with the reference move, and the number of exact matches determined the fitness.

Parameter	**Value**
Number of runs	10
Population size	100
Generations per run	75
Training samples	4000
Test samples	4000
Stage 1 search	One ply
Fitness target	Exact match with recorded move

Table 4.1 Main Stage 1 experiment settings.

The test set was not used for selection. It was used only to evaluate the best organism of each generation. This makes the test accuracy a more useful indication of whether the evolved parameters generalize beyond the training positions.

4.1.2 Stage 2 Setup

Stage 2 was evaluated using four self-play experiments. The experiments differed mainly in the search budget used during self-play. Two experiments used fixed search depth, and two experiments used a time budget per game. All four experiments used 50 generations.

Experiment	Search budget	Generations	Population
Depth 2	Fixed depth 2	50	10
Depth 3	Fixed depth 3	50	10
10s/game	Time-based, 10 seconds per game	50	10
30s/game	Time-based, 30 seconds per game	50	10

Table 4.2 Stage 2 self-play experiments.

The Stage 2 fitness values are relative. They measure how well an organism performed against the opponents used in the self-play tournament, including Hall of Fame opponents. Therefore, a higher Stage 2 fitness value does not directly mean that the organism is stronger in an absolute sense. It means that it performed better in the particular tournament environment of that generation.

4.2 Stage 1 Results

4.2.1 Training Behaviour

The Stage 1 runs showed consistent behaviour across the ten independent seeds. The test accuracy increased during the early part of the runs and then gradually stabilized. The improvement was modest. This suggests that the genetic algorithm was able to find parameter settings that matched recorded moves better than the initial population, but that exact move prediction remained a difficult target in the chosen experimental setup.

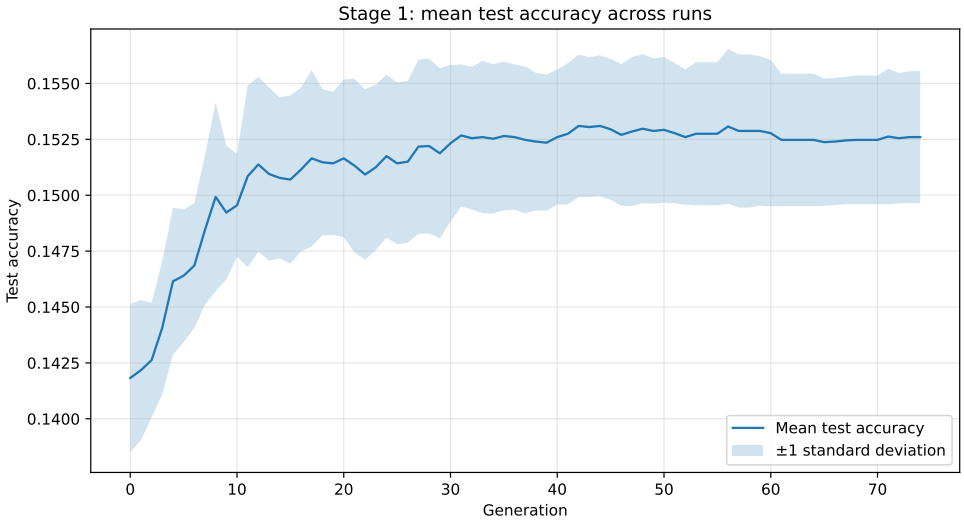


Figure 4.1 Stage 1 test accuracy over generations, averaged over ten independent runs.

The exact values require some care. The fitness function rewards only exact agreement with the move played in the recorded game. This is a strict criterion. A position may contain several reasonable moves, and a candidate evaluation function may choose a move that is playable but different from the recorded move. In addition, the recorded move is a human move. Although the games were played by strong players, the move is not guaranteed to be the objectively best move in the position, especially in fast-paced games where decisions are made under significant time pressure.

4.2.2 Test Accuracy

The final test accuracy was stable but modest. Across the ten Stage 1 runs, the mean final test accuracy was approximately 15.26%. The best observed test accuracy over all runs was approximately 15.85%. All values in Table 4.3 use the same type of metric: the percentage of positions in which one-ply move selection agrees exactly with the recorded move.

Result	Positions	Move agreement
This thesis, mean final	Test	15.26%
This thesis, best observed	Test	15.85%
David et al., best observed	Training	32.4%
David et al., best observed	Test	30.4%

Table 4.3 Stage 1 move-prediction accuracy compared with the reference work. All rows use one-ply move selection.

The results show that the Stage 1 implementation in this thesis did not reproduce the move-prediction accuracy reported by David et al. [1]. In the reference work, the best evolved Stage 1 organism solved 32.4% of the training

positions and 30.4% of the held-out test positions using one-ply search. In this thesis, the best observed Stage 1 test accuracy was 15.85%.

There are several possible reasons for this difference. First, the experimental setup is smaller: this thesis uses 4000 training positions and 75 generations, while the reference work uses 5000 training positions and reports results after 200 generations. Second, the chess engine and evaluation function used here are simplified. Third, the training data is different. The reference work uses a database of classical grandmaster games, while this thesis uses mostly fast-paced games involving strong grandmasters. Such games can contain more time-pressure decisions and more volatile move choices, which may make exact move prediction harder.

The results therefore indicate that Stage 1 provided only a limited supervised training signal. It did not produce a strong move-prediction model by itself, but it still produced organisms that could be used as starting points for the self-play stage.

4.3 Stage 2 Results

4.3.1 Self-Play Fitness

Figure 4.2 shows the best fitness value in each generation for the four Stage 2 experiments.

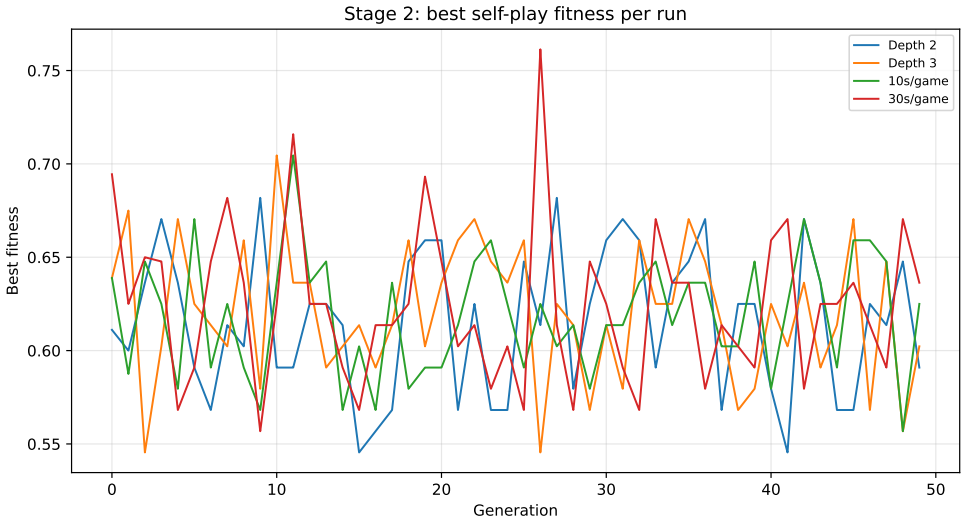


Figure 4.2 Best Stage 2 fitness over generations for the four self-play experiments.

The curves fluctuate because Stage 2 fitness is relative to the current tournament environment. Unlike Stage 1 test accuracy, it is not measured against a fixed set of reference positions. The opponents change as the population evolves, and Hall of Fame opponents also affect the evaluation. For this reason, temporary peaks in fitness should not be interpreted as permanent or absolute improvements in playing strength.

The best observed fitness was highest in the 30s/game experiment. This run reached a best observed fitness of approximately 0.7614. The depth 3 and

10s/game experiments both reached approximately 0.7045, while the depth 2 experiment reached approximately 0.6818.

Experiment	Final best fitness	Best observed fitness	Runtime
Depth 2	0.5909	0.6818	5.3 h
Depth 3	0.6023	0.7045	61.2 h
10s/game	0.6250	0.7045	49.0 h
30s/game	0.6364	0.7614	161.4 h

Table 4.4 Summary of the four Stage 2 self-play experiments.

The final best fitness values were lower than the best observed values in all four runs. This is expected in a stochastic self-play setting. A high-performing organism can appear in one generation and later be replaced or become less dominant as the population changes. The Hall of Fame mechanism reduces this problem by preserving pressure from earlier strong organisms, but it does not guarantee that the final-generation organism is the strongest checkpoint found during the run.

The search budget also had a strong effect on computational cost. The depth 2 run was much faster than the other runs, while the 30s/game run was by far the most expensive. This limited the number of independent Stage 2 repetitions that could be performed.

4.4 Final Organism Selection

The Stage 2 results alone were not sufficient to choose the final organism. The final-generation checkpoint is a natural candidate, because it is the result of the complete evolutionary run. However, in a stochastic self-play setting, the best organism does not necessarily appear in the final generation. Populations can drift, and relative fitness values depend on the opponents present in a particular generation. For this reason, both final-generation checkpoints and peak-fitness checkpoints were considered.

Six candidate organisms were selected. For each of the three stronger Stage 2 runs, two checkpoints were considered: the best organism from the final generation and the organism with the highest observed fitness peak during the run. The depth 2 run was not included in this final comparison because it used the shallowest search budget and had weaker Stage 2 results than the other completed runs.

Candidate	Source
Depth 3 final	Final-generation checkpoint from the depth 3 run
Depth 3 peak	Peak-fitness checkpoint from the depth 3 run
10s/game final	Final-generation checkpoint from the 10s/game run
10s/game peak	Peak-fitness checkpoint from the 10s/game run
30s/game final	Final-generation checkpoint from the 30s/game run
30s/game peak	Peak-fitness checkpoint from the 30s/game run

Table 4.5 Candidate organisms considered for final selection.

The selection tournament had two rounds. In the first round, the peak-fitness organism from each run was matched against the final-generation organism from the same run. This produced one representative candidate from each run. In the second round, the three winners from the first round were compared against each other.

The tournament games were played through the engine's UCI interface in Arena. Arena was used as a tournament interface and opening-book provider; the moves themselves were selected by the tested engines. The candidates were tested with a fixed-depth search setting. This was chosen deliberately: the goal of the tournament was to compare the evolved evaluation functions, not the behaviour of the time-management system. Under time constraints, the result can be influenced by additional factors such as iterative deepening, time-per-move allocation, and implementation overhead. A fixed-depth setting removes these factors and makes the comparison more directly focused on the evaluation parameters.

Round	Match	Winner	Score
1	Depth 3 peak vs. Depth 3 final	Depth 3 peak	5.5/10
1	10s/game peak vs. 10s/game final	10s/game peak	7.5/10
1	30s/game final vs. 30s/game peak	30s/game final	6.0/10
2	Depth 3 peak vs. 10s/game peak	Depth 3 peak	7.0/10
2	Depth 3 peak vs. 30s/game final	Depth 3 peak	6.5/10
2	10s/game peak vs. 30s/game final	Drawn match	5.0/10

Table 4.6 Two-round tournament used to select the final organism.

The winner of the final selection tournament was the peak-fitness organism from the depth 3 Stage 2 run. This organism came from generation 10 and was selected as the final organism of the thesis. Although the 30s/game run achieved the highest observed Stage 2 fitness during training, the additional fixed-depth tournament did not select it as the strongest candidate. This supports the decision not to rely only on Stage 2 training fitness when choosing the final organism.

As an additional observation, a time-based match was also played between the selected depth 3 peak organism and the final organism from the 30s/game run. In that setting, the 30s/game final organism won by a considerable margin. This result was not used as the main selection criterion, because the goal of the final tournament was to compare evaluation functions under a fixed-depth setting. However, it suggests that the search budget used during Stage 2 training can affect practical play under similar testing conditions. An organism evolved under time-based search may be better adapted to time-based games, while an organism selected under fixed-depth testing may better reflect the strength of the evaluation function itself.

4.5 Final Organism

4.5.1 Selected Organism

The selected final organism is the peak-fitness checkpoint from generation 10 of the depth 3 Stage 2 run. It represents a set of evolved evaluation parameters

obtained after Stage 1 initialization and Stage 2 self-play refinement. Its selection is based on a separate tournament against other candidate checkpoints, rather than only on the fitness value recorded during its own training run.

A subset of the decoded parameters of the final organism is shown in Table 4.7. The pawn value is fixed at 100 centipawns in the implementation, while the other values and feature weights are evolved.

Parameter	**Decoded value**
Pawn value	100
Knight value	202
Bishop value	419
Rook value	648
Queen value	963
Bishop pair bonus	23
Tempo bonus	45
Passed pawn bonus	63
Connected passed pawn bonus	63
Pawn shield bonus	53
Knight square multiplier	61

Table 4.7 Selected decoded parameters of the final organism.

The decoded values should be interpreted as parameters of this particular evaluation formula, not as independent chess knowledge. For example, the knight value is much lower than the bishop value, but this does not mean that the evolved engine treats a knight as literally worth half a bishop in all positions. Other terms, such as knight-square bonuses, mobility, pawn structure, and king-safety features, also contribute to the final score. The table is therefore useful for documenting the selected organism, but the individual values should not be interpreted in isolation.

4.5.2 Move-Prediction Accuracy

As an additional diagnostic evaluation, the selected final organism was tested on the held-out Stage 1 test set. This test measures how often the move selected by the engine agrees exactly with the recorded move.

The final organism was evaluated in two settings. First, it was evaluated with one-ply move selection, matching the Stage 1 evaluation setting. In this setting, it correctly predicted 611 moves out of 4000 test positions, corresponding to a move agreement of 15.28%. This is close to the mean final Stage 1 test accuracy of 15.26%, but below the best observed Stage 1 test accuracy of 15.85%. This indicates that self-play refinement did not lead to higher agreement with recorded moves than the supervised Stage 1 training when measured with the same one-ply metric.

Second, the final organism was evaluated with fixed-depth 3 search. In this setting, it correctly predicted 955 moves out of 4000 test positions, corresponding to a move agreement of 23.88%. This result is not directly comparable to the Stage 1 one-ply metric, because it includes the effect of search. However, it better reflects how the final organism is used as part of the chess engine. The improvement

from 15.28% to 23.88% suggests that search helps the evolved evaluation function select moves that agree more often with the recorded moves.

Evaluator	Setting	Move agreement
Mean final Stage 1	One-ply move selection	15.26%
Best observed Stage 1	One-ply move selection	15.85%
Final organism	One-ply move selection	15.28%
Final organism	Fixed-depth 3 search	23.88%

Table 4.8 Move-prediction accuracy of the selected final organism under different evaluation settings.

The one-ply result is useful for comparison with Stage 1, while the fixed-depth result shows how the selected organism behaves when combined with search. The two values therefore answer different questions: the first measures the raw evaluator as a one-ply move predictor, while the second measures the selected organism as part of the final chess engine.

4.6 Discussion and Limitations

The experiments show that the two-stage evolutionary pipeline can produce working evaluation parameters for the implemented chess engine. Stage 1 produced modest move-prediction improvement from recorded games and provided organisms suitable for self-play initialization. Stage 2 then allowed these organisms to compete and evolve further under different search budgets. Finally, the additional selection tournament provided a way to choose a final organism from several plausible candidates.

The results also have several limitations. The first limitation is the Stage 1 fitness function. Exact move prediction is a strict and imperfect measure of evaluation quality. A move different from the recorded move is counted as wrong even if it is also reasonable. Conversely, matching a recorded move does not guarantee that the evaluator understands the position well. This limits how much can be concluded from Stage 1 accuracy alone.

The second limitation concerns the size of the Stage 1 improvement. The supervised stage produced only a small improvement in move-prediction accuracy and did not reproduce the results reported by David et al. [1]. One possible reason is the difference in data. The reference work used positions from games of grandmasters rated above 2600 Elo, while this thesis used positions from mostly fast-paced games involving strong grandmasters. Fast time controls involve stronger time pressure, and the recorded moves may therefore be less stable as training targets. Other possible causes include the smaller dataset, fewer generations, different feature implementation, different parameter ranges, and the simplified chess engine used in this thesis.

The third limitation is the relative nature of Stage 2 fitness. Self-play fitness measures performance against the current tournament opponents, not absolute chess strength. A high fitness value in one generation can reflect dominance over the current population rather than general playing ability. Hall of Fame opponents

reduce this problem, but they do not remove it completely. The final selection tournament was added partly because of this limitation.

The fourth limitation is computational cost. Stage 2 becomes expensive quickly as the search budget increases. This restricts the number of independent repetitions that can be performed and makes it harder to distinguish stable trends from random variation.

A further limitation is that only the evaluation function is evolved, while the surrounding search remains relatively simple. The engine does not include several advanced search techniques used in stronger chess engines, such as quiescence search, transposition tables, or sophisticated move ordering. As a result, the practical playing strength of the final organism is limited not only by the evolved evaluation parameters, but also by the search procedure that uses them.

These limitations affect how the experimental results should be read. The experiments demonstrate that the implemented pipeline can evolve and compare candidate evaluation functions, but they do not establish that the resulting organism is a strong chess engine in an absolute sense. The results should therefore be understood as an evaluation of the evolutionary tuning method within the simplified engine environment used in this thesis.

Conclusion

This thesis explored the use of genetic algorithms for evolving the numerical parameters of a chess evaluation function. The goal was to implement a chess engine with a parameterized evaluation function and to test whether evolutionary methods can automatically tune its weights. The method was inspired by a two-stage approach: first, organisms were evolved using positions extracted from recorded games, and then selected organisms were refined through self-play.

The implementation provides the full experimental pipeline. It includes a chess engine core, a fixed hand-engineered evaluation function, a genome representation for evaluation parameters, genetic operators, Stage 1 training from recorded games, Stage 2 self-play evolution, checkpointing, metrics output, and interactive testing through a console mode and the Universal Chess Interface. With this setup, it was possible to run repeatable experiments, store evolved organisms, compare different search budgets, and select a final organism from the generated checkpoints.

The Stage 1 experiments showed that the supervised move-prediction task produced only modest improvement. Across ten independent runs, the mean final test accuracy was 15.26%, and the best observed test accuracy was 15.85%. These results did not reach the Stage 1 accuracy reported by David et al. [1]. As discussed in Chapter 4, one possible contributing factor is the difference in data: the dataset used in this thesis consisted mostly of fast-paced games involving very strong grandmasters, while the reference work used a different database of grandmaster games. Other differences in the experimental setup, engine implementation, and number of generations may also have affected the result.

The Stage 2 experiments showed that self-play evolution can further explore the parameter space, but that its fitness values must be interpreted carefully. Self-play fitness is relative to the current tournament environment and does not directly measure absolute playing strength. Four self-play experiments were performed with different search budgets: fixed depth 2, fixed depth 3, 10 seconds per game, and 30 seconds per game. The 30s/game run reached the highest observed Stage 2 fitness, but it was also the most computationally expensive. Because final-generation checkpoints are not guaranteed to be the strongest organisms found during self-play, a separate tournament was used for final organism selection.

The final organism selected in this thesis was the peak-fitness checkpoint from generation 10 of the depth 3 Stage 2 run. It was chosen because it won the fixed-depth selection tournament against the other candidate checkpoints. When evaluated on the held-out move-prediction test set, it achieved 15.28% agreement with recorded moves using one-ply move selection and 23.88% agreement using fixed-depth 3 search. The one-ply result shows that self-play refinement did not improve the original supervised move-prediction metric. However, the higher fixed-depth result suggests that the evolved evaluation function should be interpreted together with search, which is how the final engine is intended to operate.

The experiments demonstrate that the implemented system can evolve, evaluate, store, and compare chess evaluation functions. They also show the main difficulties in applying genetic algorithms to chess evaluation. Exact move prediction is a strict training signal, self-play fitness is noisy and relative, and the final playing strength depends not only on the evolved evaluation parameters but also

on the search procedure that uses them. The resulting organism should therefore be interpreted as the result of an experimental evolutionary tuning pipeline rather than as a competitive chess engine.

Future Work

The system could be extended in three main directions. The most natural extension would be to evolve not only the evaluation function, but also parts of the search procedure. The reference method followed in this thesis includes a second evolutionary stage in which search-related parameters are also evolved. In this thesis, the search algorithm remains fixed and only the evaluation parameters are tuned. Extending the genome to include search parameters would allow organisms to differ not only in their evaluation weights, but also in how they guide the search.

A second direction would be to improve the set of tunable evaluation features. The evaluation function used in this thesis is hand-engineered and intentionally limited. It includes several common chess concepts, such as material, mobility, pawn structure, and king safety, but it is not comparable to the evaluation functions used in strong chess engines. A richer and more carefully designed feature set, together with better parameter ranges, could give evolution a more expressive search space and make the evolved weights easier to interpret.

A third improvement would be to use a larger and more stable dataset. The dataset used in this thesis consisted mostly of fast-paced games involving very strong grandmasters. Although these games are useful, fast time controls can lead to more volatile move choices. A database based mainly on classical games between grandmasters, or on positions validated by a strong engine, could provide more reliable training targets for the Stage 1 move-prediction task.

Bibliography

1. David, Omid E.; Herik, H. Jaap van den; Koppel, Moshe; Netanyahu, Nathan S. Genetic Algorithms for Evolving Computer Chess Programs. *IEEE Transactions on Evolutionary Computation*. 2014, vol. 18, no. 5, pp. 779–789. ISSN 1089-778X, ISSN 1941-0026. Available from DOI: 10.1109/TEVC.2013.2285111.
2. Shannon, Claude E. Programming a Computer for Playing Chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*. 1950, vol. 41, no. 314, pp. 256–275. ISSN 1941-5982, ISSN 1941-5990. Available from DOI: 10.1080/14786445008521796.
3. Campbell, Murray; Hoane, A.Joseph; Hsu, Feng-hsiung. Deep Blue. *Artificial Intelligence*. 2002, vol. 134, no. 1–2, pp. 57–83. ISSN 00043702. Available from DOI: 10.1016/S0004-3702(01)00129-1.
4. Eiben, Agoston E.; Smith, James E. *Introduction to Evolutionary Computing*. 2. edition. Heidelberg New York Dordrecht London: Springer, 2015. Natural Computing Series. ISBN 978-3-662-44873-1.
5. Goldberg, David E. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, Mass: Addison-Wesley Pub. Co, 1989. ISBN 978-0-201-15767-3.
6. Koza, John R. *Genetic Programming*. Cambridge, Mass.: MIT Press, 1992. Complex Adaptive Systems. ISBN 978-0-262-52791-0.
7. Russell, Stuart J.; Norvig, Peter. *Artificial Intelligence: A Modern Approach*. Fourth edition. Hoboken: Pearson, 2021. Pearson Series in Artificial Intelligence. ISBN 978-0-13-461099-3.
8. Knuth, Donald E.; Moore, Ronald W. An Analysis of Alpha-Beta Pruning. *Artificial Intelligence*. 1975, vol. 6, no. 4, pp. 293–326. ISSN 00043702. Available from DOI: 10.1016/0004-3702(75)90019-3.
9. Edwards, Steven J. *Portable Game Notation Specification and Implementation Guide* [online]. [visited on 2026-07-11]. Available from: https://www.saremba.de/chessgml/standards/pgn/pgn-complete.htm.
10. *Chessprogramming Wiki: Forsyth-Edwards Notation* [online]. [visited on 2026-07-11]. Available from: https://www.chessprogramming.org/Forsyth-Edwards_Notation.
11. *Shredder Chess: UCI Protocol* [online]. [visited on 2026-07-11]. Available from: https://www.shredderchess.com/chess-features/uci-universal-chess-interface.html.
12. The Stockfish developers. *Introducing NNUE Evaluation* [online]. Stockfish, 2020-08-07. [visited on 2026-06-27]. Available from: https://stockfishchess.org/blog/2020/introducing-nnue-evaluation/.
13. Tunstall-Pedoe, William. Genetic Algorithms Optimizing Evaluation Functions. *ICGA Journal*. 1991, vol. 14, no. 3, pp. 119–128. ISSN 24682438, ISSN 13896911. Available from DOI: 10.3233/ICG-1991-14303.

14. Kendall, G.; Whitwell, G. An Evolutionary Approach for the Tuning of a Chess Evaluation Function Using Population Dynamics. In: *Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546)*. Seoul, South Korea: IEEE, 2001, vol. 2, pp. 995–1002. ISBN 978-0-7803-6657-2. Available from DOI: 10.1109/CEC.2001.934299.

15. Baxter, Jonathan; Tridgell, Andrew; Weaver, Lex. Learning to Play Chess Using Temporal Differences. *Machine Learning*. 2000, vol. 40, no. 3, pp. 243–263. ISSN 08856125. Available from DOI: 10.1023/A:1007634325138.

16. Fogel, D.B.; Hays, T.J.; Hahn, S.L.; Quon, J. A Self-Learning Evolutionary Chess Program. *Proceedings of the IEEE*. 2004, vol. 92, no. 12, pp. 1947–1954. ISSN 0018-9219. Available from DOI: 10.1109/JPROC.2004.837633.

17. Hauptman, Ami; Sipper, Moshe. Using Genetic Programming to Evolve Chess Endgame Players. In: *Genetic Programming*. Berlin, Heidelberg: Springer, 2005, vol. 3447, pp. 120–131. Lecture Notes in Computer Science. ISBN 978-3-540-25436-2 978-3-540-31989-4. Available from DOI: 10.1007/978-3-540-31989-4_11.

18. David-Tabibi, Omid; Koppel, Moshe; Netanyahu, Nathan S. Expert-Driven Genetic Algorithms for Simulating Evaluation Functions. *Genetic Programming and Evolvable Machines*. 2011, vol. 12, no. 1, pp. 5–22. ISSN 1389-2576, ISSN 1573-7632. Available from DOI: 10.1007/s10710-010-9103-4.

19. David-Tabibi, Omid; Van Den Herik, H. Jaap; Koppel, Moshe; Netanyahu, Nathan S. Simulating Human Grandmasters: Evolution and Coevolution of Evaluation Functions. In: *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation*. Montreal Québec Canada: ACM, 2009, pp. 1483–1490. ISBN 978-1-60558-325-9. Available from DOI: 10.1145/1569901.1570100.

List of Figures

1 A position with equal material but a positional advantage for White. 7

4.1 Stage 1 test accuracy over generations, averaged over ten independent runs. 34
4.2 Best Stage 2 fitness over generations for the four self-play experiments. 35

List of Tables

2.1 Main feature groups used by the evaluation function. 18

4.1 Main Stage 1 experiment settings. 32
4.2 Stage 2 self-play experiments. 33
4.3 Stage 1 move-prediction accuracy compared with the reference work. All rows use one-ply move selection. 34
4.4 Summary of the four Stage 2 self-play experiments. 36
4.5 Candidate organisms considered for final selection. 36
4.6 Two-round tournament used to select the final organism. 37
4.7 Selected decoded parameters of the final organism. 38
4.8 Move-prediction accuracy of the selected final organism under different evaluation settings. 39

List of Abbreviations

FEN Forsyth–Edwards Notation, a text representation of a chess position.

NNUE Efficiently updatable neural network, a neural-network evaluation method used in modern chess engines.

PGN Portable Game Notation, a standard text format for storing chess games.

SAN Standard Algebraic Notation, a standard notation for writing chess moves.

UCI Universal Chess Interface, a protocol for communication between a chess engine and a graphical user interface.

A Attachments

A.1 Electronic Attachments

The electronic attachments to this thesis are available in the public GitHub repository:

`https://github.com/Inval1dka/chessgp`

The repository contains the complete source code of the implemented application, including the C# projects, tests, plotting script, and documentation. It also contains the data files used for the experiments, selected experiment artifacts, and license information for the third-party data source.